COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010	**SECTION:**	CPE21S1
DATE PERFORMED:	07/30/26	**DATE SUBMITTED:**	
NAME:	Reyes, Eli Justin L.	**INSTRUCTOR:**	Engr. Jimlord Quejado

ACTIVITY NO. 4.1 **Hands-on Activity 4.1 Stacks**

1. PROCEDURE OUTPUT

ILO A

Input

```cpp
#include <iostream>
#include <stack> // Calling stack from the STL


int main() {
    // Create an object for the stack
    std::stack<int> newStack;

    std::cout << "Stack Empty? " << newStack.empty() << std::endl;
    // Push to the stack
    newStack.push(3);
    newStack.push(8);
    newStack.push(15);

    std::cout << "Stack Empty? " << newStack.empty() << std::endl;
    std::cout << "The size of the stack: " << newStack.size() << std::endl;
    std::cout << "The top of the element is: " << newStack.top() << std::endl;
    std::cout << "Removing elements from the stack: \n";
    newStack.pop();
    std::cout << "The top of the element is: " << newStack.top() << std::endl;
    newStack.pop();
    std::cout << "The top of the element is: " << newStack.top() << std::endl;
    newStack.pop();
    std::cout << "Stack Empty? " << newStack.empty() << std::endl;

    return 0;
}
```

Output

```
The size of the stack: 3
The top of the element is: 15
Removing elements from the stack:
The top of the element is: 8
The top of the element is: 3
Stack Empty? 1
PS C:\Users\TIPQC\Documents\CPE 010 - CPE21S1 (REYES)\HOA_4.1>
```

Table 4-1. Output of ILO A

ILO B

B.1

INPUT

```cpp
#include <iostream>

// Static Array, what do we need?
const size_t maxCap = 100;
int stack[maxCap];
int top = -1, i, newData;

void push();
void pop();
void Top();
bool isEmpty();
bool isFull();
void displayStack();

int main() {
    int choice;

    std::cout << "Enter the number of max elements for the new stack: ";
    std::cin >> i;

    while (true) {
        std::cout << "Stack Operations" << std::endl;
        std::cout << " 1. PUSH \n 2. POP \n 3. TOP \n 4. isEmpty \n 5. DISPLAY STACK" << std::endl;
        std::cin >> choice;
        switch (choice) {
            case 1: push();
            break;
            case 2: pop();
            break;
            case 3: Top();
            break;
            case 4: std::cout << isEmpty() << std::endl;
            break;
            case 5: displayStack();
            break;
            default: std::cout << "Invalid Choice!" << std::endl;
            break;
        }
    }
```

```
40
41      return 0;
42  }
43
44  bool isEmpty() {
45      // Create a condition to check whether the Top element is equal to -1
46      if (top == -1) return true;
47      return false;
48  }
49
50  bool isFull() {
51      if (top >= i-1) return true;
52      return false;
53  }
54
55  void push() {
56      // Check if full -> if yes, return an error
57      if (isFull()) {
58          std::cout << "Stack Overflow!" << std::endl;
59          return;
60      }
61
62      std::cout << "New Value to add into the stack: ";
63      std::cin >> newData;
64      stack[++top] = newData;
65  }
66
67  void Top() {
68      // Check if the stack is empty
69      if (isEmpty()) {
70          std:: cout << "Stack Underflow!" << std::endl;
71      }
72
73      std::cout << "The element on the top of the stack is " << stack[top] << std::endl;
74  }
75
```

```
76  void pop() {
77      // Check if the stack is empty
78      if (isEmpty()) {
79          std::cout << "Popping: " << stack[top] << std::endl;
80          top--;
81      }
82  }
83
84  void displayStack() {
85      if (isEmpty()) {
86          std::cout << "Stack Empty" << std::endl;
87          return;
88      }
89      for (int j = top; j >= 0; j--) {
90          if (j < top) {
91              std::cout << ", ";
92          }
93          std::cout << stack[j];
94      }
95      std::cout << std::endl;
96  }
```

OUTPUT

```
Enter the number of max elements for the new stack: 3
Stack Operations
 1. PUSH
 2. POP
 3. TOP
 4. isEmpty
 5. DISPLAY STACK
1
New Value to add into the stack: 3
Stack Operations
 1. PUSH
 2. POP
 3. TOP
 4. isEmpty
 5. DISPLAY STACK
2
Stack Operations
 1. PUSH
 2. POP
 3. TOP
 4. isEmpty
 5. DISPLAY STACK
3
The element on the top of the stack is 3
```

```
Stack Operations
 1. PUSH
 2. POP
 3. TOP
 4. isEmpty
 5. DISPLAY STACK
4
0
Stack Operations
 1. PUSH
 2. POP
 3. TOP
 4. isEmpty
 5. DISPLAY STACK
```

Pushing 3 Elements and using displayStack

```
Enter the number of max elements for the new stack: 3
```

```
New Value to add into the stack: 2
Stack Operations
 1. PUSH
 2. POP
 3. TOP
 4. isEmpty
 5. DISPLAY STACK
1
New Value to add into the stack: 1
Stack Operations
 1. PUSH
 2. POP
 3. TOP
 4. isEmpty
 5. DISPLAY STACK
5
1, 2, 3
```

Table 4-2. Output of ILO B.1

Code Analysis (Conclusion)	I have built a solid, functional menu-driven stack application in C++ using a static array, successfully implementing the core LIFO structure and a dynamic displayStack() output that lists elements from top-to-bottom without a trailing comma. However, my pop() function condition is currently backward (if (isEmpty())), meaning it only attempts to remove items when the stack is empty, which causes annoying errors to pop up crashing the program at stack[-1]. My Top() function doesn't have a return value; statement after its underflow warning, it might attempt to print an empty stack, also my lack of input validation on the user-defined size "i" allows inputs higher than maxCap = 100, creating a dangerous buffer overflow vulnerability.

B.2

INPUT

```cpp
#include <iostream>
#include "stackList.h"

int main() {

    std::cout << "--- PUSHING 'c' 'p' 'e' ONTO THE STACK --- \n";

    // Push
    push('c');
    push('p');
    push('e');

    // Display the current top
    Top<char>(); // Top should be 'e'

    displayAll<char>();

    std::cout << "\n --- Popping all the elements --- \n";

    while (!isEmpty<char>()) {
        std::cout << "Popped: " << pop<char>() << std::endl;
    }

    std::cout << "\n --- Testing the Underflow --- \n";
    pop<char>();

    return 0;
}
```

```cpp
#ifndef STACK_LIST_H
#define STACK_LIST_H

// Class node that contains data and the next
template <typename T>
class Node {
    public:
    T data; // Value that we need to store
    Node* next; // Pointer to the next node
};

// Global pointer for the top of the stack
template <typename T>
Node<T> *head = nullptr;

// Operations for the stack

// Push
template <typename T>
void push (T newData) {
    // Create a new node and assign the newData
    Node<T>* newNode  = new Node<T>;
    newNode -> data = newData;
    // New node points to current top, then becomes the new top
    newNode -> next = head<T>;
    head<T> = newNode;
}
// Pop
template <typename T>
T pop() {
    // Check if the stack is empty
    if (head<T> == nullptr) {
        std::cout << "Stack Overflow. " << std::endl;
    }

    // Save the current top node
    Node<T>* temp = head<T>;
    // Store its data
    T tempVal = temp -> data;
    // Move the head to the next node
```

```
40      // Move the head to the next node
41      head<T> = head<T> -> next;
42      // Free the removed node
43      delete temp;
44      // Return the removed value
45      return tempVal;
46  }
47  // Top
48  template <typename T>
49  void Top() {
50      // Error Catching:
51      if (head<T> == nullptr) {
52          std::cout << "Stack Underflow. " << std::endl;
53          return;
54      }
55      // Display the Top node
56      std::cout << "Top of Stack: " << head<T> -> data << std::endl;
57  }
58  // isEmpty
59  template <typename T>
60  bool isEmpty() {
61      return head<T> == nullptr;
62  }
63
64  template <typename T>
65  void displayAll() {
66      if (isEmpty<T>()) {
67          std::cout << "Stack is empty! \n";
68          return;
69      }
70
71      std::cout << "Stack (top to bottom): ";
72      Node<T>* current = head<T>;
73      while (current != nullptr) {
74          std::cout << current -> data;
75          if (current -> next != nullptr) std::cout << " -> ";
76          current = current -> next;
77      }
```

```
78      std::cout << std::endl;
79  }
80
81
82  #endif
```

OUTPUT

```
PS C:\Users\TIPQC\Documents\CPE 010 - CPE21S1 (REYES)\HOA_4.1\ILO_B.2>
icrosoft-MIEngine-Out-pjswhaqt.22h' '--stderr=Microsoft-MIEngine-Error-
--- PUSHING 'c' 'p' 'e' ONTO THE STACK ---
Top of Stack: e
Stack (top to bottom): e -> p -> c

 --- Popping all the elements ---
Popped: e
Popped: p
Popped: c

 --- Testing the Underflow ---
Stack Overflow.
```

Table 4-3. Output of ILO B.2.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

ILO C

INPUT

```cpp
#include <iostream>
#include <string>

const size_t maxCap = 100;
char stack[maxCap];
int posStack[maxCap];
int top = -1, i;
bool errorFound = false;

bool isEmpty();
bool isFull();
void push(char symbol, int position);
void push();
void pop();
void Top();
void displayStack();
void checkExpression();

int main() {
    int choice;

    std::cout << "Enter the number of max elements for the new stack: ";
    std::cin >> i;

    while (true) {
        std::cout << "\nStack Operations" << std::endl;
        std::cout << " 1. PUSH \n 2. POP \n 3. TOP \n 4. isEmpty \n 5. DISPLAY STACK \n 6. CHECK EXPRESSION FOR BALANCE \n 7. EXIT" << std::endl;
        std::cin >> choice;
        switch (choice) {
            case 1: push();
            break;
            case 2: pop();
            break;
            case 3: Top();
            break;
            case 4: std::cout << isEmpty() << std::endl;
            break;
            case 5: displayStack();
            break;
            case 6: checkExpression();
            break;
```

```
48      return 0;
49  }
50
51  bool isEmpty() {
52      if (top == -1) return true;
53      return false;
54  }
55
56  bool isFull() {
57      if (top >= i - 1) return true;
58      return false;
59  }
60
61  char matchingOpen(char closeSymbol) {
62      if (closeSymbol == ')') return '(';
63      if (closeSymbol == ']') return '[';
64      return '{';
65  }
66
67  char matchingClose(char openSymbol) {
68      if (openSymbol == '(') return ')';
69      if (openSymbol == '[') return ']';
70      return '}';
71  }
72
73  // push(symbol, position) contains ALL of the balance-checking logic.
74  // position is the index of this character in the original expression,
75  // used only to make the error messages specific. Pass -1 when there's
76  // no meaningful position (e.g. manual single-character push from the menu).
77  void push(char symbol, int position) {
78      if (symbol != '(' && symbol != ')' &&
79          symbol != '{' && symbol != '}' &&
80          symbol != '[' && symbol != ']') {
81          return;
82      }
83
84      if (symbol == '(' || symbol == '{' || symbol == '[') {
85          if (isFull()) {
86              std::cout << "Stack Overflow!" << std::endl;
```

```
88              return;
89          }
90          stack[++top] = symbol;
91          posStack[top] = position;
92          return;
93      }
94
95      // Closing symbol
96      if (isEmpty()) {
97          std::cout << "Error";
98          if (position >= 0) std::cout << " at position " << position;
99          std::cout << ": found closing symbol '" << symbol
100                    << "' but the stack is empty (nothing to match it with)." << std::endl;
101         errorFound = true;
102         return;
103     }
104
105     char popped = stack[top];
106     int poppedPos = posStack[top];
107     top--;
108
109     if (symbol != matchingClose(popped)) {
110         std::cout << "Error";
111         if (position >= 0) std::cout << " at position " << position;
112         std::cout << ": closing symbol '" << symbol << "' does not match '" << popped << "'";
113         if (poppedPos >= 0) std::cout << " (opened at position " << poppedPos << ")";
114         std::cout << " -- expected '" << matchingClose(popped) << "'." << std::endl;
115         errorFound = true;
116     }
117 }
118
119 void push() {
120     if (isFull()) {
121         std::cout << "Stack Overflow!" << std::endl;
122         return;
123     }
124
125     std::cout << "New symbol to push: ";
126     char newData;
127     std::cin >> newData;
128     push(newData, -1);
```

```cpp

void pop() {
    if (isEmpty()) {
        std::cout << "Stack Underflow!" << std::endl;
        return;
    }
    std::cout << "Popping: " << stack[top] << std::endl;
    top--;
}

void Top() {
    if (isEmpty()) {
        std::cout << "Stack Underflow!" << std::endl;
        return;
    }
    std::cout << "The element on the top of the stack is " << stack[top] << std::endl;
}

void displayStack() {
    if (isEmpty()) {
        std::cout << "Stack Empty" << std::endl;
        return;
    }
    for (int j = top; j >= 0; j--) {
        if (j < top) {
            std::cout << ", ";
        }
        std::cout << stack[j];
    }
    std::cout << std::endl;
}

void checkExpression() {
    std::string expr;
    std::cout << "Enter an expression to check: ";
    std::cin.ignore();
    std::getline(std::cin, expr);

    top = -1;
    errorFound = false;
```

```cpp
void checkExpression() {
    std::string expr;
    std::cout << "Enter an expression to check: ";
    std::cin.ignore();
    std::getline(std::cin, expr);

    top = -1;
    errorFound = false;

    for (int position = 0; position < (int)expr.size(); position++) {
        push(expr[position], position);
        if (errorFound) break;
    }

    if (!errorFound && !isEmpty()) {
        std::cout << "Error: expression ended with unmatched opening symbol(s):" << std::endl;
        for (int j = top; j >= 0; j--) {
            std::cout << "  position " << posStack[j] << ": '" << stack[j] << "' was never closed" << std::endl;
        }
        errorFound = true;
    }

    std::cout << std::endl;
    std::cout << "Expression: " << expr << std::endl;
    std::cout << "Result: " << (errorFound ? "INVALID (N)" : "VALID (Y)") << std::endl;
}
```

ARRAY

```
void checkExpression() {
    std::string expr;
    std::cout << "Enter an expression to check: ";
    std::cin.ignore();
    std::getline(std::cin, expr);

    while (top != nullptr) {
        Node* temp = top;
        top = top->next;
        delete temp;
    }
    count = 0;
    errorFound = false;

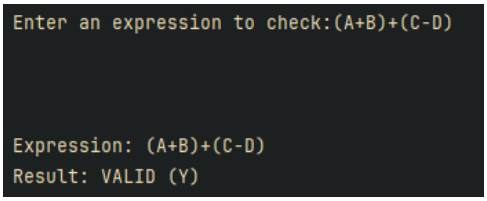
    for (int position = 0; position < (int)expr.size(); position++) {
        push(expr[position], position);
        if (errorFound) break;
    }

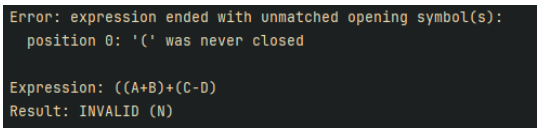
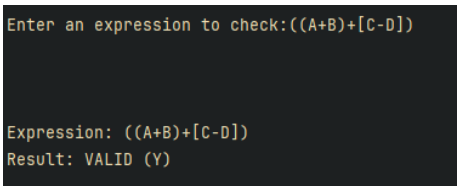
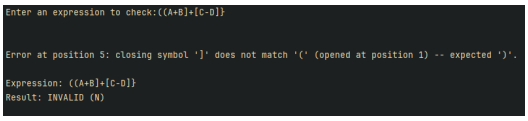
    if (!errorFound && !isEmpty()) {
        std::cout << "Error: expression ended with unmatched opening symbol(s):" << std::endl;
        Node* cur = top;
        while (cur != nullptr) {
            std::cout << "  position " << cur->position << ": '" << cur->data << "' was never closed" << std::endl;
            cur = cur->next;
        }
        errorFound = true;
    }

    std::cout << std::endl;
    std::cout << "Expression: " << expr << std::endl;
    std::cout << "Result: " << (errorFound ? "INVALID (N)" : "VALID (Y)") << std::endl;
}
```

LINKED LIST

OUTPUT

Expression	Valid? (Y/N)	Output (Console Screenshot)	Analysis
(A+B)+(C-D)	Y		Balanced equally by both sides

((A+B)+(C-D)	N		Not balanced due to missing parentheses in the end
((A+B)+[C-D])	Y		Balanced, both starting and ending with parentheses
((A+B]+[C-D]}	N		Not balanced (due to curly brace)

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **Gemini**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Comments (for fix)**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **I only made it wrote comments on what causes error (later then removed)**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.